



JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING
TATHAWADE, PUNE-33
(An Autonomous Institute Affiliated to Savitribai Phule Pune
University,Pune)



LAB MANUAL
OF
Data structures and algorithms LAB
(IT3102)
S.Y.B.Tech(2024)
INFORMATION TECHNOLOGY

Authors: Dr.Priya Karemore

© JSPM Group of Institutes, Pune. All Rights Reserved. All the information in this Course Manual is confidential. Participants shall refrain from copying, distributing, misusing/ or disclosing the content to any third parties any circumstances whatsoever.

2.3 Laboratory Plan

Sr. No.	TITLE OF ASSIGNMENT
1.	Write a C program to Implement Stack data structure using an Array and Linked list.
2.	Write a C program to reverse a string using stack/ perform balancing of parenthesis/ perform expression evaluation.
3.	Write a C program to Implement Queue data structure using an Array and Linked list.
4.	Create a Binary Search Tree,take input from user and perform following operations on it. a. Insertion of a node in a tree b. Deletion of a node c. Searching of a node d. Display by using any one traversal method.
5.	Write a C Program to Implement AVL Tree.
6.	Write a C program to Implement Adjacency Matrix representation of Graph data structure.
7.	Write a C Program using Breadth First Traversal for a user defined Graph and Depth First Traversal for a user defined Graph.
8.	Write a C program to implement a hash table on names for telephone directory data.

Text Books: T1. Horowitz, Sahani, Dinesh Mehta, “Fundamentals of Data Structures in C++” Galgotia Publisher, ISBN: 8175152788, 9788175152786.

T2. M Folk, B Zoellick, G. riccardi, “File Structures, Pearson Education”, ISBN: 81-7758-37-5

T3. Peter Brass, “Advanced Data Structures”, Cambridge University Press, ISBN: 978-1-107-43982-5

Reference Books:

R1. A. Aho, J. Hpcroft, J. Ulman, “Data Structures and Algorithms”, Pearson Education, 1998, ISBN-0-201-43578-0

R2. Michael J Folk, “File Structures an Object Oriented approach with C++”, Pearson education, ISBN:81-7758-373-5

R3. Sartaj Sahani, "Data Structures, Algorithms and Applications in C++", Second Edition, University press, ISBN: 81-7371522 X.

R4. G A V Pai, "Data Structures and Algorithms", McGraw Hill Companies, ISBN- 9780070667266

R5. Goodrich, Tamassia, Goldwasser, "Data Structures and Algorithms in Java", Willey Publication, ISBN:9788126551903.

RS COEFF

EXPERIMENT NO – 1

Title:

Implement Stack Data Structure using Array and Linked List

Aim:

To implement the stack data structure using both arrays and linked lists in C.

Objective:

1. To understand the stack data structure and its properties (LIFO: Last In, First Out).
 2. To implement the stack using arrays and linked lists.
 3. To perform basic stack operations: push, pop, and peek.
 4. To compare the advantages of using arrays versus linked lists for stack implementation.
-

Problem Statement:

Write a C program to implement a stack data structure using an array and a linked list. The program should support the following operations:

1. **Push:** Add an element to the top of the stack.
 2. **Pop:** Remove the topmost element from the stack.
 3. **Peek:** Retrieve the topmost element without removing it.
 4. **Display:** Show all elements in the stack.
-

Theory:

Stack is a linear data structure that follows **LIFO (Last In First Out) Principle**, the last element inserted is the first to be popped out. It means both insertion and deletion operations happen at one end only.

LIFO (Last In First Out) Principle

Here are some real world examples of LIFO

- Consider a stack of plates. When we add a plate, we add at the top. When we remove, we remove from the top.

- A **shuttlecock box** (or any other box that is closed from one end) is another great real-world example of the **LIFO (Last In, First Out)** principle where do insertions and removals from the same end.

Types of Stack:

- **Fixed Size Stack** : As the name suggests, a fixed size stack has a fixed size and cannot grow or shrink dynamically. If the stack is full and an attempt is made to add an element to it, an overflow error occurs. If the stack is empty and an attempt is made to remove an element from it, an underflow error occurs.
- **Dynamic Size Stack** : A dynamic size stack can grow or shrink dynamically. When the stack is full, it automatically increases its size to accommodate the new element, and when the stack is empty, it decreases its size. This type of stack is implemented using a linked list, as it allows for easy resizing of the stack.

Stack Operations:

1. **Push**: Adds an element to the stack.

Adds an item to the stack. If the stack is full, then it is said to be an **Overflow condition**.

Algorithm for Push Operation:

- Before pushing the element to the stack, we check if the stack is **full** .
- If the stack is full (**top == capacity-1**) , then **Stack Overflows** and we cannot insert the element to the stack.
- Otherwise, we increment the value of top by 1 (**top = top + 1**) and the new value is inserted at **top position** .
- The elements can be pushed into the stack till we reach the **capacity** of the stack.

2. **Pop**: Removes the topmost element.

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an **Underflow condition**.

Algorithm for Pop Operation:

- Before popping the element from the stack, we check if the stack is **empty** .
- If the stack is empty (**top == -1**), then **Stack Underflows** and we cannot remove any element from the stack.

- Otherwise, we store the value at top, decrement the value of top by 1 (**top = top – 1**) and return the stored top value.
3. **Peek:** Retrieves the topmost element without modifying the stack.

Returns the top element of the stack.

Algorithm for Top Operation:

- Before returning the top element from the stack, we check if the stack is empty.
- If the stack is empty (top == -1), we simply print “Stack is empty”.
- Otherwise, we return the element stored at **index = top**.

Array-Based Stack:

- Uses a fixed-size array to store stack elements.
- Requires managing a top variable to keep track of the stack's topmost position.

Linked List-Based Stack:

- Uses dynamic memory allocation for nodes.
- Each node contains data and a pointer to the next node.
- Does not require a predefined size, allowing flexibility.

Code:

1. Stack Implementation Using Array:

```
#include <stdio.h>
#define SIZE 5 // Define the maximum size of the stack
struct Stack {
    int arr[SIZE];
    int top;
};
// Initialize the stack
void initializeStack(struct Stack *stack) {
    stack->top = -1;
}
// Check if the stack is empty
```

```
int isEmpty(struct Stack *stack) {
    return stack->top == -1;
}

// Check if the stack is full
int isFull(struct Stack *stack) {
    return stack->top == SIZE - 1;
}

// Push an element onto the stack
void push(struct Stack *stack, int value) {
    if (isFull(stack)) {
        printf("Stack Overflow! Cannot push %d.\n", value);
        return;
    }
    stack->arr[++stack->top] = value;
    printf("%d pushed onto the stack.\n", value);
}

// Pop an element from the stack
int pop(struct Stack *stack) {
    if (isEmpty(stack)) {
        printf("Stack Underflow! Cannot pop.\n");
        return -1;
    }
    return stack->arr[stack->top--];
}

// Display the stack
void display(struct Stack *stack) {
    if (isEmpty(stack)) {
        printf("Stack is empty!\n");
        return;
    }
    printf("Stack elements: ");
    for (int i = 0; i <= stack->top; i++) {
```

```

        printf("%d ", stack->arr[i]);
    }
    printf("\n");
}

int main() {
    struct Stack stack;
    initializeStack(&stack);
    push(&stack, 10);
    push(&stack, 20);
    push(&stack, 30);
    display(&stack);
    printf("Popped: %d\n", pop(&stack));
    display(&stack);
    return 0;
}

```

2. Stack Implementation Using Linked List:

```

#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node *next;
};

// Push an element onto the stack
void push(struct Node **top, int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Heap Overflow! Cannot push %d.\n", value);
        return;
    }
}

```



```

    }

    newNode->data = value;
    newNode->next = *top;
    *top = newNode;
    printf("%d pushed onto the stack.\n", value);
}

// Pop an element from the stack
int pop(struct Node **top) {
    if (*top == NULL) {
        printf("Stack Underflow! Cannot pop.\n");
        return -1;
    }
    struct Node *temp = *top;
    int poppedValue = temp->data;
    *top = (*top)->next;
    free(temp);
    return poppedValue;
}

// Display the stack
void display(struct Node *top) {
    if (top == NULL) {
        printf("Stack is empty!\n");
        return;
    }
    printf("Stack elements: ");
    struct Node *temp = top;
    while (temp) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```

```
int main() {  
    struct Node *top = NULL;  
    push(&top, 10);  
    push(&top, 20);  
    push(&top, 30);  
    display(top);  
    printf("Popped: %d\n", pop(&top));  
    display(top);  
    return 0;  
}
```

Complexity Analysis of Operations on Stack Data Structure:

Operations	Time Complexity	Space Complexity
push()	O(1)	O(1)
pop()	O(1)	O(1)
top() or peek()	O(1)	O(1)

Conclusion:

1. A stack can be implemented using either arrays or linked lists.
2. **Array-Based Stack** is simple but has a fixed size, making it less flexible.
3. **Linked List-Based Stack** provides dynamic memory allocation, which allows the stack to grow or shrink as needed.
4. Understanding these implementations helps in choosing the right approach based on application requirements.

RSCOEIT

EXPERIMENT NO – 2

Practical Title : Write a C program to reverse a string using stack/ perform balancing of parenthesis/ perform expression evaluation.

Aim: To reverse a string using stack/ perform balancing of parenthesis/ perform expression evaluation.

Objective:

1. To understand how a stack can be used to reverse a string.
 2. To implement the stack operations (push and pop) to reverse a string.
 3. To reinforce the concept of LIFO (Last In, First Out) using stack.
-

Problem Statement:

Write a C program to reverse a string using a stack. The program should use the push and pop operations to reverse the string efficiently.

Theory:

A **stack** is a data structure that follows the LIFO principle (Last In, First Out). When reversing a string using a stack:

1. Push each character of the string onto the stack.
2. Pop the characters from the stack, which retrieves them in reverse order.

This is because the last character pushed onto the stack is the first one retrieved when popping.

Code:

```
#include <stdio.h>

#include <string.h>

#define MAX 100 // Maximum size of the stack

// Stack structure
struct Stack {
    char arr[MAX];
```

```
    int top;
};
// Initialize the stack
void initializeStack(struct Stack *stack) {
    stack->top = -1;
}
// Check if the stack is full
int isFull(struct Stack *stack) {
    return stack->top == MAX - 1;
}
// Check if the stack is empty
int isEmpty(struct Stack *stack) {
    return stack->top == -1;
}
// Push a character onto the stack
void push(struct Stack *stack, char ch) {
    if (isFull(stack)) {
        printf("Stack Overflow! Cannot push '%c'.\n", ch);
        return;
    }
    stack->arr[++stack->top] = ch;
}
// Pop a character from the stack
char pop(struct Stack *stack) {
    if (isEmpty(stack)) {
        printf("Stack Underflow! Cannot pop.\n");
        return '\0';
    }
    return stack->arr[stack->top--];
}

// Function to reverse a string using stack
```

```
void reverseString(char str[]) {  
    struct Stack stack;  
    initializeStack(&stack);  
    // Push all characters of the string onto the stack  
    for (int i = 0; i < strlen(str); i++) {  
        push(&stack, str[i]);  
    }  
    // Pop characters from the stack to reverse the string  
    for (int i = 0; i < strlen(str); i++) {  
        str[i] = pop(&stack);  
    }  
}  
  
int main() {  
    char str[MAX];  
  
    // Input the string  
    printf("Enter a string: ");  
    fgets(str, MAX, stdin);  
  
    // Remove newline character from input if present  
    str[strcspn(str, "\n")] = '\0';  
  
    printf("Original string: %s\n", str);  
  
    // Reverse the string using stack  
    reverseString(str);  
  
    // Display the reversed string  
    printf("Reversed string: %s\n", str);  
  
    return 0;  
}
```

Explanation:

Stack Operations:

1. The stack is implemented using an array of char with a maximum size of 100.
2. The push function adds characters to the stack.
3. The pop function retrieves characters in reverse order.

Reversing the String:

1. Each character of the string is pushed onto the stack.
2. When characters are popped, they are placed back into the string, effectively reversing it.

String Handling:

1. The fgets function is used to read a string, ensuring it handles spaces.
2. strcspn removes the trailing newline character, if present.

Sample Input/Output:

Input:

Enter a string: hello

Output:

Original string: hello

Reversed string: olleh

Input:

Enter a string: Data Structures

Output:

Original string: Data Structures

Reversed string: serutcurtS ataD

Conclusion:

1. A stack can effectively reverse a string by utilizing the LIFO property.
2. This program demonstrates how the push and pop operations can be used for string manipulation.
3. Understanding the stack's behavior provides insight into solving similar problems efficiently.

RSCOEIT

+

EXPERIMENT NO – 3

Practical Title: Implement Queue Data Structure Using Array and Linked List

Aim: To implement the queue data structure using both arrays and linked lists in C.

Objective:

1. To understand the queue data structure and its FIFO (First In, First Out) property.
 2. To implement queue operations such as enqueue, dequeue, and display.
 3. To compare the array-based and linked list-based implementations of a queue.
-

Problem Statement:

Write a C program to implement the queue data structure using both an array and a linked list. The program should support the following operations:

1. **Enqueue:** Add an element to the rear of the queue.
 2. **Dequeue:** Remove an element from the front of the queue.
 3. **Display:** Show all elements in the queue.
-

Theory:

Queue Overview:

A **queue** is a linear data structure that follows the FIFO principle.

- **Enqueue:** Adds an element to the rear of the queue.
- **Dequeue:** Removes an element from the front of the queue.
- **Display:** Shows the elements in the queue from front to rear.

Array-Based Implementation:

- Uses a fixed-size array to store queue elements.
- Requires managing front and rear indices.

Linked List-Based Implementation:

- Dynamically allocates memory for each node.
- Each node stores data and a pointer to the next node.
- No fixed size, allowing the queue to grow or shrink dynamically.

Code:

1. Queue Implementation Using Array:

```
#include <stdio.h>

#define SIZE 5 // Maximum size of the queue

struct Queue {
    int arr[SIZE];
    int front, rear;
};

// Initialize the queue
void initializeQueue(struct Queue *queue) {
    queue->front = -1;
    queue->rear = -1;
}

// Check if the queue is empty
int isEmpty(struct Queue *queue) {
    return queue->front == -1;
}

// Check if the queue is full
int isFull(struct Queue *queue) {
    return queue->rear == SIZE - 1;
}
```

// Enqueue operation

```
void enqueue(struct Queue *queue, int value) {  
    if (isFull(queue)) {  
        printf("Queue Overflow! Cannot enqueue %d.\n", value);  
        return;  
    }  
    if (isEmpty(queue)) queue->front = 0;  
    queue->arr[++queue->rear] = value;  
    printf("%d enqueued to the queue.\n", value);  
}
```

// Dequeue operation

```
int dequeue(struct Queue *queue) {  
    if (isEmpty(queue)) {  
        printf("Queue Underflow! Cannot dequeue.\n");  
        return -1;  
    }  
    int value = queue->arr[queue->front];  
    if (queue->front == queue->rear) {  
        queue->front = queue->rear = -1; // Reset the queue  
    } else {  
        queue->front++;  
    }  
    return value;  
}
```

// Display the queue

```
void display(struct Queue *queue) {  
    if (isEmpty(queue)) {  
        printf("Queue is empty!\n");  
        return;  
    }
```

```

    }
    printf("Queue elements: ");
    for (int i = queue->front; i <= queue->rear; i++) {
        printf("%d ", queue->arr[i]);
    }
    printf("\n");
}

```

```

int main() {
    struct Queue queue;
    initializeQueue(&queue);

    enqueue(&queue, 10);
    enqueue(&queue, 20);
    enqueue(&queue, 30);

    display(&queue);

    printf("Dequeued: %d\n", dequeue(&queue));
    display(&queue);

    return 0;
}

```

2. Queue Implementation Using Linked List:

```

#include <stdio.h>
#include <stdlib.h>

// Node structure for the queue
struct Node {
    int data;

```

```
    struct Node *next;
};

// Structure for the queue
struct Queue {
    struct Node *front, *rear;
};

// Initialize the queue
void initializeQueue(struct Queue *queue) {
    queue->front = queue->rear = NULL;
}

// Check if the queue is empty
int isEmpty(struct Queue *queue) {
    return queue->front == NULL;
}

// Enqueue operation
void enqueue(struct Queue *queue, int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Heap Overflow! Cannot enqueue %d.\n", value);
        return;
    }
    newNode->data = value;
    newNode->next = NULL;
    if (queue->rear == NULL) {
        queue->front = queue->rear = newNode;
    } else {
        queue->rear->next = newNode;
        queue->rear = newNode;
    }
}
```

```
}  
printf("%d enqueued to the queue.\n", value);  
}  
  
// Dequeue operation  
int dequeue(struct Queue *queue) {  
    if (isEmpty(queue)) {  
        printf("Queue Underflow! Cannot dequeue.\n");  
        return -1;  
    }  
    struct Node *temp = queue->front;  
    int value = temp->data;  
    queue->front = queue->front->next;  
    if (queue->front == NULL) queue->rear = NULL;  
    free(temp);  
    return value;  
}
```

```
// Display the queue  
void display(struct Queue *queue) {  
    if (isEmpty(queue)) {  
        printf("Queue is empty!\n");  
        return;  
    }  
    printf("Queue elements: ");  
    struct Node *temp = queue->front;  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
    printf("\n");  
}
```

```
int main() {  
    struct Queue queue;  
    initializeQueue(&queue);  
  
    enqueue(&queue, 10);  
    enqueue(&queue, 20);  
    enqueue(&queue, 30);  
  
    display(&queue);  
  
    printf("Dequeued: %d\n", dequeue(&queue));  
    display(&queue);  
  
    return 0;  
}
```

Sample Input/Output:

Array-Based Queue:

10 enqueued to the queue.
20 enqueued to the queue.
30 enqueued to the queue.
Queue elements: 10 20 30
Dequeued: 10
Queue elements: 20 30

Linked List-Based Queue:

10 enqueued to the queue.
20 enqueued to the queue.
30 enqueued to the queue.
Queue elements: 10 20 30
Dequeued: 10
Queue elements: 20 30

Conclusion:

Array-Based Queue:

- Simple implementation.
- Limited by a fixed size.

- Efficient for small, predictable-sized datasets.

Linked List-Based Queue:

- Dynamic memory allocation makes it flexible.
- Ideal for scenarios where the size of the queue is unknown or varies frequently.

Both implementations demonstrate the FIFO property and provide a solid foundation for understanding queue data structures.

EXPERIMENT NO – 4

Practical Title: Binary Search Tree Implementation and Operations

Aim:

To implement a Binary Search Tree (BST) and perform various operations such as insertion, deletion, searching, and displaying nodes using a traversal method.

Objective:

1. To understand the structure and properties of a Binary Search Tree.
 2. To perform insertion, deletion, and searching operations in a BST.
 3. To display the elements of the tree using a traversal method (e.g., Inorder Traversal).
-

Problem Statement:

Write a C program to:

1. Create a Binary Search Tree (BST) by taking input from the user.
2. Perform the following operations:
 1. **Insertion:** Add a node to the BST.
 2. **Deletion:** Remove a node from the BST.
 3. **Searching:** Find a node in the BST.
 4. **Display:** Show the BST using Inorder Traversal.

Theory:

A **Binary Search Tree (BST)** is a hierarchical data structure with the following properties:

1. **Node Property:** Each node contains a key.
 2. **Left Subtree:** Contains nodes with keys less than the parent node's key.
 3. **Right Subtree:** Contains nodes with keys greater than the parent node's key.
 4. **Traversal Methods:**
 1. **Inorder Traversal:** Left -> Root -> Right (Displays nodes in sorted order).
 2. **Preorder Traversal:** Root -> Left -> Right.
 3. **Postorder Traversal:** Left -> Right -> Root.
-

Code Implementation:

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for the BST
struct Node {
    int data;
    struct Node *left, *right;
};

// Function to create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert a node into the BST
```

```

struct Node* insertNode(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value); // Create a new node if tree is empty
    }
    if (value < root->data) {
        root->left = insertNode(root->left, value); // Insert in the left subtree
    } else if (value > root->data) {
        root->right = insertNode(root->right, value); // Insert in the right subtree
    }
    return root;
}

```

// Find the minimum value node in the BST (helper for deletion)

```

struct Node* findMin(struct Node* root) {
    while (root->left != NULL) {
        root = root->left;
    }
    return root;
}

```

// Delete a node from the BST

```

struct Node* deleteNode(struct Node* root, int value) {
    if (root == NULL) {
        printf("Node not found!\n");
        return root;
    }
    if (value < root->data) {
        root->left = deleteNode(root->left, value); // Delete from left subtree
    } else if (value > root->data) {
        root->right = deleteNode(root->right, value); // Delete from right subtree
    } else {
        // Node with one or no child

```

```

    if (root->left == NULL) {
        struct Node* temp = root->right;
        free(root);
        return temp;
    } else if (root->right == NULL) {
        struct Node* temp = root->left;
        free(root);
        return temp;
    }
    // Node with two children
    struct Node* temp = findMin(root->right);
    root->data = temp->data; // Copy the inorder successor's data
    root->right = deleteNode(root->right, temp->data); // Delete the inorder successor
}
return root;
}

```

// Search for a node in the BST

```

struct Node* searchNode(struct Node* root, int value) {
    if (root == NULL || root->data == value) {
        return root; // Return the node if found or NULL if not found
    }
    if (value < root->data) {
        return searchNode(root->left, value); // Search in left subtree
    }
    return searchNode(root->right, value); // Search in right subtree
}

```

// Inorder Traversal (Left -> Root -> Right)

```

void inorderTraversal(struct Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
    }
}

```

```
        printf("%d ", root->data);  
        inorderTraversal(root->right);  
    }  
}
```

```
int main() {  
    struct Node* root = NULL;  
    int choice, value;  
  
    do {  
        printf("\nMenu:\n");  
        printf("1. Insert Node\n");  
        printf("2. Delete Node\n");  
        printf("3. Search Node\n");  
        printf("4. Display (Inorder Traversal)\n");  
        printf("5. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                printf("Enter value to insert: ");  
                scanf("%d", &value);  
                root = insertNode(root, value);  
                break;  
            case 2:  
                printf("Enter value to delete: ");  
                scanf("%d", &value);  
                root = deleteNode(root, value);  
                break;  
            case 3:  
                printf("Enter value to search: ");
```

```
scanf("%d", &value);

struct Node* found = searchNode(root, value);

if (found != NULL) {
    printf("Node %d found in the BST.\n", found->data);
} else {
    printf("Node not found in the BST.\n");
}

break;

case 4:
    printf("Inorder Traversal: ");
    inorderTraversal(root);
    printf("\n");
    break;

case 5:
    printf("Exiting...\n");
    break;

default:
    printf("Invalid choice! Please try again.\n");
}

} while (choice != 5);

return 0;
}
```

Sample Input/Output:

Input:

Menu:

1. Insert Node
2. Delete Node
3. Search Node
4. Display (Inorder Traversal)
5. Exit

Enter your choice: 1

Enter value to insert: 50

Enter your choice: 1

Enter value to insert: 30

Enter your choice: 1

Enter value to insert: 70

Enter your choice: 4

Output:

Inorder Traversal: 30 50 70

Conclusion:

1. A Binary Search Tree (BST) allows efficient searching, insertion, and deletion of nodes.
2. The **Inorder Traversal** of a BST displays nodes in ascending order.
3. This program demonstrates the basic operations of a BST and its practical applications in data management.

EXPERIMENT NO – 5

Practical Title: Implement AVL Tree in C

Aim:

To implement an AVL tree in C, ensuring that the tree remains height-balanced after insertion and deletion operations.

Objective:

1. To understand the concept of an AVL tree and its self-balancing property.
 2. To implement insertion and deletion in an AVL tree.
 3. To demonstrate the use of rotations (LL, RR, LR, RL) to maintain balance in the AVL tree.
-

Problem Statement:

Write a C program to implement an AVL tree that supports:

1. Insertion of nodes.
 2. Automatic height balancing using rotations.
 3. Inorder traversal to display the elements in sorted order.
-

Theory:

An **AVL Tree** is a type of self-balancing binary search tree, where the difference between the heights of the left and right subtrees (balance factor) of any node is at most 1.

Rotations:

1. **LL (Left-Left):** Single right rotation is performed.
2. **RR (Right-Right):** Single left rotation is performed.
3. **LR (Left-Right):** Left rotation followed by right rotation.
4. **RL (Right-Left):** Right rotation followed by left rotation.

Key Properties:

1. **Height Balance:** Ensures $\text{balance factor} = \text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, 1\}$
 $\text{balance factor} = \text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, 1\}$.
 2. **Time Complexity:**
 1. Search, Insertion, Deletion: $O(\log n)$.
-

Code Implementation:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure for AVL tree
```

```
struct Node {
```

```
    int key;
```

```
    struct Node* left;
```

```
    struct Node* right;
```

```
    int height;
```



```
};
```

```
// Function to get the height of a node
```

```
int getHeight(struct Node* node) {  
    if (node == NULL) return 0;  
    return node->height;  
}
```

```
// Function to calculate the balance factor of a node
```

```
int getBalanceFactor(struct Node* node) {  
    if (node == NULL) return 0;  
    return getHeight(node->left) - getHeight(node->right);  
}
```

```
// Function to create a new node
```

```
struct Node* createNode(int key) {  
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));  
    node->key = key;  
    node->left = NULL;  
    node->right = NULL;  
    node->height = 1; // New node is initially at height 1  
    return node;  
}
```

```
// Right rotation (LL Case)
```

```
struct Node* rotateRight(struct Node* y) {  
    struct Node* x = y->left;  
    struct Node* T2 = x->right;
```

```
// Perform rotation
```

```
x->right = y;  
y->left = T2;
```

```

// Update heights
y->height = 1 + (getHeight(y->left) > getHeight(y->right) ? getHeight(y->left) : getHeight(y->right));
x->height = 1 + (getHeight(x->left) > getHeight(x->right) ? getHeight(x->left) : getHeight(x->right));

return x; // New root
}

```

// Left rotation (RR Case)

```

struct Node* rotateLeft(struct Node* x) {

```

```

    struct Node* y = x->right;

```

```

    struct Node* T2 = y->left;

```

```

// Perform rotation

```

```

y->left = x;

```

```

x->right = T2;

```

```

// Update heights

```

```

x->height = 1 + (getHeight(x->left) > getHeight(x->right) ? getHeight(x->left) : getHeight(x->right));

```

```

y->height = 1 + (getHeight(y->left) > getHeight(y->right) ? getHeight(y->left) : getHeight(y->right));

```

```

return y; // New root

```

```

}

```

// Function to insert a node in AVL tree

```

struct Node* insertNode(struct Node* node, int key) {

```

```

// Perform normal BST insertion

```

```

if (node == NULL) return createNode(key);

```

```

if (key < node->key) {

```

```

    node->left = insertNode(node->left, key);

```

```

} else if (key > node->key) {

```

```

        node->right = insertNode(node->right, key);
    } else {
        return node; // Duplicate keys not allowed
    }

    // Update the height of the current node
    node->height = 1 + (getHeight(node->left) > getHeight(node->right) ? getHeight(node->left) :
getHeight(node->right));

    // Get the balance factor
    int balance = getBalanceFactor(node);

    // Perform rotations to balance the tree
    // LL Case
    if (balance > 1 && key < node->left->key) {
        return rotateRight(node);
    }
    // RR Case
    if (balance < -1 && key > node->right->key) {
        return rotateLeft(node);
    }
    // LR Case
    if (balance > 1 && key > node->left->key) {
        node->left = rotateLeft(node->left);
        return rotateRight(node);
    }
    // RL Case
    if (balance < -1 && key < node->right->key) {
        node->right = rotateRight(node->right);
        return rotateLeft(node);
    }

    return node; // Return the unchanged node pointer

```

```
}
```

```
// Inorder Traversal (Left -> Root -> Right)
```

```
void inorderTraversal(struct Node* root) {
```

```
    if (root != NULL) {
```

```
        inorderTraversal(root->left);
```

```
        printf("%d ", root->key);
```

```
        inorderTraversal(root->right);
```

```
    }
```

```
}
```

```
int main() {
```

```
    struct Node* root = NULL;
```

```
    int choice, key;
```

```
    do {
```

```
        printf("\nMenu:\n");
```

```
        printf("1. Insert Node\n");
```

```
        printf("2. Display (Inorder Traversal)\n");
```

```
        printf("3. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1:
```

```
                printf("Enter key to insert: ");
```

```
                scanf("%d", &key);
```

```
                root = insertNode(root, key);
```

```
                break;
```

```
            case 2:
```

```
                printf("Inorder Traversal: ");
```

```
                inorderTraversal(root);
```

```
        printf("\n");
        break;
    case 3:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice! Please try again.\n");
    }
} while (choice != 3);

return 0;
}
```

Sample Input/Output:

Input:

1. Insert Node
2. Display (Inorder Traversal)
3. Exit

Enter your choice: 1
Enter key to insert: 30

Enter your choice: 1
Enter key to insert: 20

Enter your choice: 1
Enter key to insert: 40

Enter your choice: 2

Output:

Inorder Traversal: 20 30 40

Conclusion:

1. An AVL tree maintains a height-balanced binary search tree using rotations.
2. Insertion and balancing are performed efficiently with $O(\log n)$ complexity.
3. The program demonstrates AVL tree operations and ensures the tree remains balanced after each insertion.

EXPERIMENT NO – 6

Practical Title : Implement Adjacency Matrix Representation of Graph

Aim:

To implement a graph data structure using the adjacency matrix representation in C.

Objective:

1. To understand the adjacency matrix representation of a graph.
2. To implement operations such as adding edges and displaying the graph.
3. To analyze how adjacency matrices are used to represent connections between graph nodes.

Problem Statement:

Write a C program to represent a graph using an adjacency matrix. The program should allow the user to:

1. Add edges between nodes.
 2. Display the adjacency matrix of the graph.
-

Theory:

Graph Representation:

1. A **graph** is a collection of vertices (nodes) and edges (connections between nodes).
 2. **Adjacency Matrix Representation:**
 1. Uses a 2D array to represent a graph.
 2. If there is an edge between vertex i and vertex j , then $matrix[i][j]=1$ otherwise $matrix[i][j]=0$.
 3. For **undirected graphs**, $matrix[i][j]=matrix[j][i]$.
 4. For **weighted graphs**, $matrix[i][j]$ stores the weight of the edge.
-

Code Implementation:

```
#include <stdio.h>

#define MAX 10 // Maximum number of vertices

// Function to initialize the adjacency matrix
void initializeMatrix(int matrix[MAX][MAX], int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            matrix[i][j] = 0; // No edges initially
        }
    }
}
```

// Function to add an edge to the graph

```
void addEdge(int matrix[MAX][MAX], int src, int dest) {  
    matrix[src][dest] = 1; // Add edge from source to destination  
    matrix[dest][src] = 1; // For undirected graph, add edge back  
}
```

// Function to display the adjacency matrix

```
void displayMatrix(int matrix[MAX][MAX], int vertices) {  
    printf("Adjacency Matrix:\n");  
    for (int i = 0; i < vertices; i++) {  
        for (int j = 0; j < vertices; j++) {  
            printf("%d ", matrix[i][j]);  
        }  
        printf("\n");  
    }  
}
```

int main() {

```
    int matrix[MAX][MAX]; // Adjacency matrix  
    int vertices, edges, src, dest;
```

// Input number of vertices

```
    printf("Enter the number of vertices: ");  
    scanf("%d", &vertices);
```

// Initialize the adjacency matrix

```
    initializeMatrix(matrix, vertices);
```

// Input number of edges

```
    printf("Enter the number of edges: ");  
    scanf("%d", &edges);
```



```
// Input edges
for (int i = 0; i < edges; i++) {
    printf("Enter source and destination for edge %d (0-based indexing): ", i + 1);
    scanf("%d %d", &src, &dest);
    addEdge(matrix, src, dest);
}

// Display the adjacency matrix
displayMatrix(matrix, vertices);

return 0;
}
```

Sample Input/Output:

Input:

Enter the number of vertices: 4
Enter the number of edges: 4
Enter source and destination for edge 1 (0-based indexing): 0 1
Enter source and destination for edge 2 (0-based indexing): 0 2
Enter source and destination for edge 3 (0-based indexing): 1 2
Enter source and destination for edge 4 (0-based indexing): 2 3

Output:

Adjacency Matrix:
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0

Explanation of Code:

Initialization:

1. The adjacency matrix is initialized with zeros, indicating no edges initially.

Adding Edges:

1. Each edge is represented by setting `matrix[src][dest] = 1`.
2. For an undirected graph, the reverse edge is also added (`matrix[dest][src] = 1`).

Displaying the Matrix:

1. The 2D array is displayed row by row, showing the connections between vertices.
-

Conclusion:

1. Adjacency matrices provide a simple and efficient way to represent graphs, especially for dense graphs.
2. The space complexity is $O(V^2)$, where V is the number of vertices.
3. This program demonstrates how to represent and visualize a graph using adjacency matrices, which can be used for graph algorithms like BFS, DFS, and shortest path calculations.

EXPERIMENT NO – 7

Practical Title : Breadth-First Traversal (BFS) and Depth-First Traversal (DFS) for a User-Defined Graph

Aim:

To implement Breadth-First Traversal (BFS) and Depth-First Traversal (DFS) for a user-defined graph in C.

Objective:

1. To understand the concepts of BFS and DFS in graph traversal.
 2. To implement BFS using a queue and DFS using recursion.
 3. To traverse all vertices of a user-defined graph and display the traversal order.
-

Problem Statement:

Write a C program to input a user-defined graph and perform the following traversals:

1. **Breadth-First Traversal (BFS)** using a queue.
 2. **Depth-First Traversal (DFS)** using recursion.
-

Theory:

Graph Traversal:

Breadth-First Traversal (BFS):

1. Traverses the graph level by level.
2. Uses a **queue** to visit nodes.
3. Suitable for finding the shortest path in an unweighted graph.

Depth-First Traversal (DFS):

1. Traverses the graph by exploring as far as possible along each branch before backtracking.
2. Uses a **stack** (implicit via recursion) for traversal.
3. Useful for exploring all paths in a graph.

Adjacency Matrix Representation:

A **graph** is represented using an adjacency matrix where:

- $\text{matrix}[i][j]=1$ $\text{matrix}[i][j] = 1$ indicates an edge between vertex i and vertex j .
 - $\text{matrix}[i][j]=0$ $\text{matrix}[i][j] = 0$ indicates no edge.
-

Code Implementation:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 10 // Maximum number of vertices

int adjMatrix[MAX][MAX]; // Adjacency matrix
int visited[MAX]; // Visited array to track visited vertices
int queue[MAX]; // Queue for BFS
int front = -1, rear = -1;

// Function to add an edge to the graph
void addEdge(int src, int dest) {
    adjMatrix[src][dest] = 1;
    adjMatrix[dest][src] = 1; // For undirected graph
}

// Initialize the adjacency matrix and visited array
void initializeGraph(int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            adjMatrix[i][j] = 0;
        }
        visited[i] = 0;
    }
}

// Breadth-First Traversal (BFS)
void bfs(int startVertex, int vertices) {
    front = rear = -1; // Initialize queue
    queue[++rear] = startVertex;
    visited[startVertex] = 1;
```

```

printf("Breadth-First Traversal: ");
while (front != rear) {
    int currentVertex = queue[++front];
    printf("%d ", currentVertex);

    for (int i = 0; i < vertices; i++) {
        if (adjMatrix[currentVertex][i] == 1 && !visited[i]) {
            queue[++rear] = i;
            visited[i] = 1;
        }
    }
}
printf("\n");
}

```

// Depth-First Traversal (DFS) using recursion

```

void dfs(int vertex, int vertices) {
    printf("%d ", vertex);
    visited[vertex] = 1;

    for (int i = 0; i < vertices; i++) {
        if (adjMatrix[vertex][i] == 1 && !visited[i]) {
            dfs(i, vertices);
        }
    }
}

```

```

int main() {
    int vertices, edges, src, dest, startVertex;

    // Input the number of vertices and edges
    printf("Enter the number of vertices: ");

```

```
scanf("%d", &vertices);

printf("Enter the number of edges: ");

scanf("%d", &edges);


// Initialize the graph
initializeGraph(vertices);


// Input the edges
for (int i = 0; i < edges; i++) {
    printf("Enter source and destination for edge %d (0-based indexing): ", i + 1);
    scanf("%d %d", &src, &dest);
    addEdge(src, dest);
}


// Input the starting vertex for traversal
printf("Enter the starting vertex for traversal: ");
scanf("%d", &startVertex);


// Perform BFS
bfs(startVertex, vertices);


// Reset visited array for DFS
for (int i = 0; i < vertices; i++) {
    visited[i] = 0;
}


// Perform DFS
printf("Depth-First Traversal: ");
dfs(startVertex, vertices);
printf("\n");

return 0;
```

}

Sample Input/Output:

Input:

Enter the number of vertices: 5
Enter the number of edges: 6
Enter source and destination for edge 1 (0-based indexing): 0 1
Enter source and destination for edge 2 (0-based indexing): 0 2
Enter source and destination for edge 3 (0-based indexing): 1 2
Enter source and destination for edge 4 (0-based indexing): 1 3
Enter source and destination for edge 5 (0-based indexing): 2 4
Enter source and destination for edge 6 (0-based indexing): 3 4
Enter the starting vertex for traversal: 0

Output:

Breadth-First Traversal: 0 1 2 3 4
Depth-First Traversal: 0 1 2 4 3

Explanation of Code:

Adjacency Matrix Initialization:

- Initializes the graph with no edges using a $\text{vertices} \times \text{vertices}$ matrix.

Adding Edges:

- Updates the adjacency matrix for each edge.

BFS Implementation:

- Uses a queue to traverse the graph level by level.
- Marks nodes as visited and adds them to the queue for traversal.

DFS Implementation:

- Uses recursion to explore each node fully before backtracking.

Traversal Order:

- BFS explores nodes closest to the starting vertex first.
 - DFS explores as deep as possible along each path before backtracking.
-

Conclusion:

1. **Breadth-First Traversal (BFS):** Explores all neighbors of a node before moving deeper.
2. **Depth-First Traversal (DFS):** Explores as far as possible before backtracking.
3. This program demonstrates how BFS and DFS can be implemented using adjacency matrices for graph traversal.

RSCOE

EXPERIMENT NO – 8

Practical Title:

Implement Hash Table for Telephone Directory Data

Aim:

To implement a hash table to store and retrieve names in a telephone directory using C.

Objective:

1. To understand the concept of hash tables for efficient data storage and retrieval.
 2. To implement basic hash table operations such as insertion, search, and display.
 3. To handle collisions using separate chaining with linked lists.
-

Problem Statement:

Write a C program to implement a hash table to store and manage names in a telephone directory. The program should:

1. Insert a name into the hash table.
 2. Search for a name in the hash table.
 3. Display the contents of the hash table.
-

Theory:

Hash Table:

- A **hash table** is a data structure that maps keys to values using a hash function.
 - **Hash Function:** Computes an index for a key in the hash table.
 - **Collision Handling:** When two keys hash to the same index:
 - **Separate Chaining:** Store keys in a linked list at the hashed index.
-

Code Implementation:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define TABLE_SIZE 10 // Define the size of the hash table
```

```
// Node structure for the linked list (used for separate chaining)
```

```
struct Node {
```

```
    char name[50]; // Name field
```

```

    struct Node* next;
};

// Hash table array of pointers to Node
struct Node* hashTable[TABLE_SIZE];

// Hash function
int hashFunction(char* name) {
    int hash = 0;
    for (int i = 0; name[i] != '\0'; i++) {
        hash += name[i];
    }
    return hash % TABLE_SIZE;
}

// Insert a name into the hash table
void insert(char* name) {
    int index = hashFunction(name);
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    strcpy(newNode->name, name);
    newNode->next = NULL;

    if (hashTable[index] == NULL) {
        hashTable[index] = newNode; // Insert directly if no collision
    } else {
        struct Node* temp = hashTable[index];
        while (temp->next != NULL) {
            temp = temp->next; // Traverse to the end of the list
        }
        temp->next = newNode; // Add new node at the end
    }

    printf("Inserted: %s\n", name);
}

```

```
}
```

```
// Search for a name in the hash table
```

```
void search(char* name) {
```

```
    int index = hashFunction(name);
```

```
    struct Node* temp = hashTable[index];
```

```
    while (temp != NULL) {
```

```
        if (strcmp(temp->name, name) == 0) {
```

```
            printf("Name found: %s\n", name);
```

```
            return;
```

```
        }
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("Name not found: %s\n", name);
```

```
}
```

```
// Display the contents of the hash table
```

```
void display() {
```

```
    for (int i = 0; i < TABLE_SIZE; i++) {
```

```
        printf("Index %d: ", i);
```

```
        struct Node* temp = hashTable[i];
```

```
        if (temp == NULL) {
```

```
            printf("NULL");
```

```
        } else {
```

```
            while (temp != NULL) {
```

```
                printf("%s -> ", temp->name);
```

```
                temp = temp->next;
```

```
            }
```

```
        }
```

```
        printf("NULL\n");
```

```
    }
```

```
}
```

```
int main() {
```

```
    // Initialize the hash table to NULL
```

```
    for (int i = 0; i < TABLE_SIZE; i++) {
```

```
        hashTable[i] = NULL;
```

```
    }
```

```
    int choice;
```

```
    char name[50];
```

```
    do {
```

```
        printf("\nMenu:\n");
```

```
        printf("1. Insert Name\n");
```

```
        printf("2. Search Name\n");
```

```
        printf("3. Display Hash Table\n");
```

```
        printf("4. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1:
```

```
                printf("Enter name to insert: ");
```

```
                scanf("%s", name);
```

```
                insert(name);
```

```
                break;
```

```
            case 2:
```

```
                printf("Enter name to search: ");
```

```
                scanf("%s", name);
```

```
                search(name);
```

```
                break;
```

```
            case 3:
```

```
        display();
        break;
    case 4:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice! Please try again.\n");
    }
} while (choice != 4);

return 0;
}
```

Sample Input/Output:

Input:

Menu:

1. Insert Name
2. Search Name
3. Display Hash Table
4. Exit

Enter your choice: 1

Enter name to insert: Alice

Output:

Inserted: Alice

Menu:

1. Insert Name
2. Search Name
3. Display Hash Table
4. Exit

Enter your choice: 1

Enter name to insert: Bob

Inserted: Bob

Menu:

1. Insert Name
2. Search Name
3. Display Hash Table
4. Exit

Enter your choice: 3

Index 0: NULL

Index 1: NULL

Index 2: Alice -> NULL
Index 3: Bob -> NULL

Explanation:

Hash Function:

- Converts a name into an index by summing ASCII values of characters modulo the table size.

Insertion:

- If the index is empty, the name is directly inserted.
- If there's a collision, the name is added to the linked list at that index.

Search:

- Traverses the linked list at the hashed index to find the name.

Display:

- Prints the entire hash table, showing all names at each index.
-

Conclusion:

1. Hash tables allow fast insertion, search, and retrieval operations.
2. Collision handling using separate chaining ensures that the table works even when multiple keys hash to the same index.
3. This program effectively demonstrates the use of hash tables for storing and managing telephone directory data.

RSCOEIT

RSCOEIT

RSCOEIT